

Exp No: 24

MODELING CLOCK-DRIVEN CONCURRENT PROCESSES USING SC_THREAD

AND WAIT() IN SYSTEMC

AIM:

To understand and implement a clock-driven process using:

- sc_clock
- SC_THREAD
- wait()
- Event sensitivity (positive edge trigger)

in SystemC.

APPARATUS/SOFTWARE REQUIRED:

- Linux OS / WSL & Ubuntu
- systemC

PROGRAM:

```
#include <systemc.h>
```

```
SC_MODULE(ClockExample) {  
    sc_in<bool> clk;
```

```
    void process() {  
        while (true) {  
            wait();  
            std::cout << "Time: " << sc_time_stamp()  
                << " - Clock Triggered" << std::endl;  
        }  
    }  
}
```

```
SC_CTOR(ClockExample) {
```

```

SC_THREAD(process);
sensitive && clk.pos();
}
};

int sc_main(int argc, char* argv[]) {

    sc_clock clock(&quot;clock&quot;, 1, SC_NS);

    ClockExample module(&quot;module&quot;);
    module.clk(clock);

    sc_start(5, SC_NS);

    return 0;
}

```

OUTPUT:

Log

Share

```

int sc_main(int argc, char* argv[]) {
    ^
    Compile done. Starting run...

    SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
    Copyright (c) 1996-2018 by all Contributors,
    ALL RIGHTS RESERVED
    Time: 0 s - Clock Triggered
    Time: 1 ns - Clock Triggered
    Time: 2 ns - Clock Triggered
    Time: 3 ns - Clock Triggered
    Time: 4 ns - Clock Triggered
    Done

```